

Parcial 1 – Liquidación de entregas en un centro de acopio de leche

Universidad del Quindío

Ingeniería de Sistemas y Computación

Programación III – Parcial 1

Docente: Julián E. Gutiérrez Posada

Integrantes: Luisa Hernández Ramírez, Isabella Hincapié Bocanegra y Camila González Restrepo

Repositorio: <https://github.com/lu03hr/Parcial-1---Centro-de-Acopio-de-Leche>

En este parcial hicimos un programa en Elixir para un centro de acopio de leche. El programa revisa las entregas de la semana, calcula cuánto se le paga a cada productor y muestra 8 reportes. Para correrlo se usa `elixir main.exs` y para las mediciones `elixir mediciones.exs`.

Parte A. Diseño

Cómo organizamos el código

Dividimos el programa en varios archivos, cada uno con una tarea:

Archivo	Para qué sirve
<code>parametros.exs</code>	Guarda los valores fijos del enunciado como atributos de módulo (tarifa, meta, bonificación, etc.)
<code>datos.exs</code>	Productores, tanques y entregas
<code>validacion.exs</code>	Revisa si cada entrega cumple las reglas
<code>liquidacion.exs</code>	Calcula cuánto se le paga a cada productor y el comprobante
<code>Reportes.exs</code>	Calcula los reportes R1 a R8 y la combinación con el centro vecino
<code>entrada.exs</code>	Convierte lo que escribe el usuario en una entrega
<code>Vista.exs</code>	Muestra los reportes en pantalla

Archivo	Para qué sirve
main.exs	Llama todo en orden
mediciones.exs	Mide tiempos con <code>:timer.tc</code>
Util2.ex	Funciones que hicimos en clase (mostrar, ordenar, ingresar)

El orden en `main.exs` es: validar las entregas, pedir la entrega adicional, liquidar a los productores, mostrar los reportes, combinar con el centro vecino y por último mostrar el comprobante.

Colecciones que usamos

- **Listas:** los productores, tanques y entregas llegan en listas y así los dejamos, porque casi siempre hay que recorrerlos completos. Para saber si un productor o un tanque existe usamos `Enum.any?` sobre la lista. Sabemos que un mapa es más rápido para buscar (lo medimos en la Parte C), pero con 10 productores la diferencia no se nota y así el código es más fácil de leer.
- **Mapas:** cada entrega y cada productor es un mapa. También usamos mapas para agrupar: `Enum.group_by` para juntar las entregas de cada productor, y `Enum.reduce` con `Map.update` para sumar los litros por día o por tanque.
- **Tuplas:** para los resultados que pueden fallar usamos `{:ok, valor}` y `{:error, motivo}`, como pide el enunciado. Las entregas rechazadas se guardan como `{entrega, motivo}`.
- **Keyword lists:** para las opciones de `ranking/2`.
- **Rango:** los días son `1..6` y se usa para validar el día y para recorrer los 6 días en `R3` y `R5`.

Estructuras de control

- **with:** en la validación. Se revisan las 5 reglas en orden y apenas una falla se detiene, por eso solo sale el primer motivo. También lo usamos para revisar el formato de la entrega adicional.
- **Cláusulas con when:** para el ajuste por grasa (`factor_calidad`), como en los ejercicios de clase.
- **if:** para las preguntas de sí o no, por ejemplo si el productor gana bonificación o si usa transporte.
- **case:** para convertir texto a número con `Integer.parse` y `Float.parse`, y para decidir qué hacer con la entrega adicional.
- **Enum y for:** para todos los recorridos. No usamos recursividad, structs, File, procesos ni `try/rescue`.

Funciones puras e impuras

Las funciones de `validacion`, `liquidacion`, `Reportes` y `entrada` son puras: reciben datos y devuelven un resultado, sin imprimir ni leer nada. Esto nos sirvió para probarlas sueltas en

iex, por ejemplo `Liquidacion.factor_calidad(2.7)`. Las impuras están en `main.exs`, que lee del teclado con `Util2.ingresar`, y en `Vista.exs`, que imprime con `Util2.mostrar`.

Explicación de R6

En R6 la calidad se mide con el porcentaje de grasa ponderado por litros: $\text{suma}(\text{grasa} \times \text{litros}) / \text{suma}(\text{litros})$. Esto puede dar un resultado distinto al promedio simple, porque el promedio simple le da el mismo peso a todas las entregas, sin importar si son de 50 litros o de 600.

En nuestros datos se ve claro con Santiago Rosas (P07):

Día	Litros	Grasa	Grasa × litros
2	50	4.8	240
3	600	2.6	1560
5	420	2.7	1134
Total	1070		2934

Promedio simple: $(4.8 + 2.6 + 2.7) / 3 = 3.367 \%$

Promedio ponderado: $2934 / 1070 = 2.742 \%$

La entrega de 50 litros tiene mucha grasa y sube el promedio simple, pero es muy poca leche. Casi toda la leche de Santiago (1020 de 1070 litros) tiene menos de 3 %. Por eso el ponderado muestra mejor la calidad real de lo que entregó. En nuestra salida, la mejor calidad la tiene Maryory Restrepo con 4.552 %.

Parte C. Investigación

Keyword lists

Una keyword list es una lista de tuplas donde el primer elemento de cada tupla es un átomo. Por ejemplo, `[campo: :neto, orden: :desc]` es lo mismo que `[{:campo, :neto}, {:orden, :desc}]`.

Se diferencian de un mapa en que:

- conservan el orden en que se escriben,
- pueden tener claves repetidas,
- las claves solo pueden ser átomos,
- para buscar una clave hay que recorrer la lista.

Por eso en Elixir se usan mucho para pasarle opciones a una función. Nosotras las usamos en `ranking/2`, que está en `Reportes.exs`:

```
ranking(liquidacion, campo: :neto, orden: :desc, desempate: :codigo)
```

Dentro de la función leemos cada opción con Keyword:

- `Keyword.fetch!(opciones, :campo)`: el campo por el que se ordena. Es obligatorio.
- `Keyword.get(opciones, :orden, :desc)`: si no se manda, ordena de mayor a menor.
- `Keyword.get(opciones, :desempate)`: un campo para desempatar. Si no se manda, no se usa.

Para ordenar usamos `Util2.ordenar`. Usamos `ranking/2` en R2, R4, R5 y R6.

Map.merge/3

Para juntar nuestros litros por día (los de R3) con los del centro vecino hicimos esta función en `Reportes.exs`:

```
def combinar_con_vecino(litros_por_dia, centro_vecino) do
  Map.merge(litros_por_dia, centro_vecino, fn _dia, litros_propios,
    litros_vecino ->
      litros_propios + litros_vecino
    end)
end
```

La función que va de último dice qué hacer cuando un día está en los dos mapas. En este caso, sumar los litros.

Investigación. Combinación con el centro vecino (Map.merge/3)

Día	Propio	Vecino	Combinado
1	2362.5	1850.5	4213.0
2	2010.0	2100.0	4110.0
3	1865.5	1640.0	3505.5
4	2675.5	-	2675.5
5	1614.5	2350.0	3964.5
6	2471.5	-	2471.5
7	-	800.0	800.0

Ingrese el código del productor para el comprobante:

Captura 1. Resultado de la combinación (con la entrega adicional P03;T2;4;320.5;3.6). El guión significa que ese día no está en ese mapa.

1. ¿Qué habría pasado con Map.merge/2? Map.merge/2 no suma. Cuando un día está en los dos mapas, deja el valor del segundo mapa y borra el del primero. Por ejemplo, el día 1 hubiera quedado en 1850.5 en vez de 4213.0, y lo mismo pasaría con los días 2, 3 y 5.

2. ¿Por qué no sería adecuado? Porque perderíamos nuestros litros de esos días. El resultado ya no sería el total de los dos centros, sino los litros del vecino.

3. ¿Qué pasa con el día 7? La suma solo se hace con los días que están en los dos mapas. El día 7 solo está en el mapa del vecino, entonces pasa igual al resultado, con 800. Lo mismo pasa con los días 4 y 6, que solo están en el nuestro. Hay que tener en cuenta que nuestro centro solo recibe del día 1 al 6, pero Map.merge no revisa eso, así que el día 7 queda en el resultado.

Mediciones con :timer.tc/1

:timer.tc ejecuta una función y devuelve cuánto se demoró en microsegundos (µs). Hicimos 3 experimentos y cada uno lo repetimos 7 veces, porque la primera vez casi siempre se demora más. Mostramos el tiempo mínimo y el promedio.

MEDICIONES CON :timer.tc/1 (7 repeticiones por caso)			
1) Buscar el productor de cada entrega (1000 vueltas sobre 84 entregas)			
Lista + Enum.any?	mín:	36659 µs	prom: 39511 µs
Mapa + Map.has_key?	mín:	12902 µs	prom: 17246 µs
2) Litros por día con 84 entregas			
Enum.group_by y luego sumar cada grupo	mín:	0 µs	prom: 190 µs
Enum.reduce con Map.update (una pasada)	mín:	0 µs	prom: 14 µs
2) Litros por día con 16800 entregas			
Enum.group_by y luego sumar cada grupo	mín:	3788 µs	prom: 5309 µs
Enum.reduce con Map.update (una pasada)	mín:	3584 µs	prom: 4227 µs
3) Construir una lista de 5000 elementos			
lista ++ [x] (al final)	mín:	172441 µs	prom: 188781 µs
[x lista] y Enum.reverse al final	mín:	204 µs	prom: 233 µs

Captura 2. Resultado de correr elixir mediciones.exs en nuestro computador.

Experimento 1: buscar un productor en una lista o en un mapa. Buscamos el productor de cada entrega 1000 veces. Con la lista y Enum.any? se demoró en promedio 39 511 µs y con un mapa y Map.has_key? 17 246 µs, menos de la mitad. Esto pasa porque en la lista hay que ir revisando productor por productor, y en el mapa se va directo al código. Aun así dejamos la lista en la validación, porque con 10 productores la diferencia es muy pequeña.

Experimento 2: sumar los litros por día con group_by o con reduce. Con solo 84 entregas los tiempos fueron tan pequeños que el mínimo salió en 0 µs y no se puede comparar bien. Por eso repetimos las entregas 200 veces (16 800 entregas). Ahí reduce con Map.update salió un poco más rápido (4 227 µs contra 5 309 µs), porque suma todo en una sola pasada, mientras que group_by primero arma las listas de cada día y después las vuelve a recorrer para sumar.

Experimento 3: agregar elementos al final o al inicio de una lista. Armamos una lista de 5000 números. Con lista ++ [x] se demoró 188 781 µs y con [x | lista] y al final Enum.reverse solo 233 µs. La diferencia es enorme porque ++ copia toda la lista cada vez

que agrega un elemento, en cambio agregar al inicio no copia nada. Por eso en `main.exe` solo usamos `++` una vez, para la entrega adicional.

Salida del programa

Corrimos `elixir main.exe` con nuestros datos. Como entrega adicional escribimos `P03;T2;4;320.5;3.6` y al final pedimos el comprobante de `P03`. En los datos hay 96 entregas: 84 válidas y 12 con errores. Con la entrega adicional quedaron 85 válidas.

Entrega adicional, R1 y R2

```
PS C:\Users\Luisa Fernanda\Documents\UNI\PROGRAMACIÓN\Programación II\Parcial-1--Centro-de-Acopia-de-Leche> elixir main.exe
Ingrese una entrega adicional
(productor;tanque;dia;litros;grasa)
o Enter para omitir: P03;T2;4;320.5;3.6
Entrega adicional aceptada.

Reporte 1. Entregas rechazadas

%{productor: "P99", tanque: "T1", dia: 2, litros: 200, grasa: 3.5} -> productor_desconocido
%{productor: "P03", tanque: "T2", dia: 3, litros: 150, grasa: 3.2} -> productor_desconocido
%{productor: "P03", tanque: "T9", dia: 1, litros: 180, grasa: 3.4} -> tanque_desconocido
%{productor: "P06", tanque: "T5", dia: 4, litros: 210, grasa: 3.3} -> tanque_desconocido
%{productor: "P08", tanque: "T1", dia: 7, litros: 210, grasa: 3.3} -> dia_invalido
%{productor: "P10", tanque: "T2", dia: 0, litros: 190, grasa: 3.6} -> dia_invalido
%{productor: "P09", tanque: "T3", dia: "3", litros: 150, grasa: 3.0} -> dia_invalido
%{productor: "P10", tanque: "T1", dia: 3, litros: 950, grasa: 3.5} -> litros_fuera_de_rango
%{productor: "P05", tanque: "T4", dia: 5, litros: 0, grasa: 3.2} -> litros_fuera_de_rango
%{productor: "P06", tanque: "T3", dia: 1, litros: 200, grasa: 16.2} -> porcentaje_invalido
%{productor: "P02", tanque: "T1", dia: 4, litros: 180, grasa: -1.0} -> porcentaje_invalido
%{productor: "P99", tanque: "T9", dia: 9, litros: 900, grasa: 20} -> productor_desconocido

Rechazos por motivo:
productor_desconocido: 3
tanque_desconocido: 2
dia_invalido: 3
litros_fuera_de_rango: 2
porcentaje_invalido: 2

Reporte 2. Ocupación de tanques (de mayor a menor)

Tanque      Litros  Capacidad  Ocupación
Tanque Sur  2738.0  3000      91.3 %
Tanque Central  3159.0  4000      79.0 %
Tanque Norte  3903.5  5000      78.1 %
Tanque Oeste  3199.0  5100      62.7 %
```

Captura 3. La entrega adicional fue aceptada. En R1 salen las 12 entregas rechazadas con su motivo y cuántas hay de cada motivo. La última entrega tiene todos los datos malos, pero solo sale `productor_desconocido` porque es la primera regla que falla. En R2 el Tanque Central sube a 3159 litros por la entrega adicional.

R3, R4 y R5

Reporte 3. Litros recibidos por día (meta: 2000 litros)

Día 1: 2362.5 L - Meta alcanzada: Sí
Día 2: 2010.0 L - Meta alcanzada: Sí
Día 3: 1865.5 L - Meta alcanzada: No
Día 4: 2675.5 L - Meta alcanzada: Sí
Día 5: 1614.5 L - Meta alcanzada: No
Día 6: 2471.5 L - Meta alcanzada: Sí

¿Se cumplió la meta todos los días? No
¿Se cumplió la meta al menos un día? Sí

Reporte 4. Liquidación de productores (por neto, de mayor a menor)

#	Productor	Litros	Entregas	Bonif.	Transp.	Neto
1	Danna Vélez	1949.5	\$3503556	\$25000	\$90000	\$3438556
2	Isabella Hincapié	1726.0	\$2978334	\$25000	\$0	\$3003334
3	Maryory Restrepo	1707.5	\$2977254	\$25000	\$0	\$3002254
4	Luisa Hernández	1521.5	\$2657502	\$25000	\$90000	\$2592502
5	Sandro Rendón	1389.5	\$2494386	\$25000	\$90000	\$2429386
6	Rony López	1338.5	\$2462976	\$0	\$90000	\$2372976
7	Camila González	1300.5	\$2339820	\$25000	\$90000	\$2274820
8	Santiago Rosas	1070.0	\$1784520	\$25000	\$0	\$1809520
9	María Durán	996.5	\$1761444	\$25000	\$54000	\$1732444
10	Omar Cortés	0.0	\$0	\$0	\$0	\$0

Reporte 5. Productor con más litros cada día

Día 1: Sandro Rendón (P04) con 632.0 L
Día 2: Isabella Hincapié (P02) con 628.0 L
Día 3: Santiago Rosas (P07) con 600.0 L
Día 4: Maryory Restrepo (P06), María Durán (P08) con 460.0 L
Día 5: Maryory Restrepo (P06) con 435.5 L
Día 6: Danna Vélez (P05) con 744.5 L

Primer lugar en más días (2 días): Maryory Restrepo (P06)

Captura 4. En R3 los días 3 y 5 no llegaron a la meta de 2000 litros (en la captura se tapan los días 5 y 6: el día 5 tiene 1614.5 L y no cumple, el día 6 tiene 2471.5 L y sí cumple). En R4 están los 10 productores ordenados por neto; Omar Cortés sale en cero porque todas sus entregas fueron rechazadas. En R5 el día 4 hay empate entre Maryory Restrepo y María Durán y salen las dos.

R6, R7 y R8

Reporte 6. Mejor calidad (grasa ponderada por litros)

Mejor calidad: Maryory Restrepo (P06) con 4.552 %

Productor	Entregas	Ponderada	Simple
Maryory Restrepo	12	4.552	3.542
Danna Vélez	11	4.281	3.773
Isabella Hincapié	14	3.807	3.421
Rony López	10	3.474	3.380
María Durán	4	3.305	3.275
Sandro Rendón	10	3.245	3.430
Camila González	9	3.222	3.322
Luisa Hernández	12	3.206	3.283
Santiago Rosas	3	2.742	3.367

Reporte 7. Totales de la semana

Total pagado por el centro: \$22655792

Litros recibidos: 12999.5 L

Costo promedio por litro: \$1743

Reporte 8. Productores con entregas en todos los tanques

P01 - Luisa Hernández
P02 - Isabella Hincapié
P03 - Camila González
P04 - Sandro Rendón
P05 - Danna Vélez
P06 - Maryory Restrepo
P09 - Rony López

Captura 5. En R6 solo salen los que tienen 3 o más entregas válidas. En R7 el centro pagó \$22 655 792 en total, unos \$1743 por litro. En R8 salen los 7 productores que entregaron en los 4 tanques.

Combinación con el centro vecino

Está en la Captura 1, en la Parte C.

Comprobante de P03

Ingrese el código del productor para el comprobante: P03			
Comprobante de pago			
Productor: Camila González (P03)			
Día	Litros	Valor	Bonificación
1	139.0	\$230184	\$0
3	169.0	\$322452	\$0
4	422.5	\$780426	\$0
5	58.5	\$111618	\$0
6	511.5	\$895140	\$25000
Litros entregados: 1300.5 L			
Total entregas: \$2339820			
Total bonificaciones: \$25000			
Descuento por transporte: \$90000			
Neto a pagar: \$2274820			

Captura 6. El comprobante muestra cada día con sus litros, el valor y la bonificación. El día 4 suma 422.5 litros, no llega a 450 y por eso no tiene bonificación. El día 6 sí llega (511.5 litros) y gana \$25 000. Como Camila usa transporte y entregó 5 días, se le descuentan $5 \times \$18\,000 = \$90\,000$.

Lo revisamos a mano: la entrega adicional vale $320.5 \times 1800 \times 1.06 = \$611\,514$ (gana el 6 % porque la grasa es 3.6). Sin la entrega adicional, Camila tenía \$1 728 306 en entregas, y con ella \$2 339 820, que es justo esa diferencia.

Pruebas con entradas inválidas

Probamos el programa con varias entradas para revisar que no se cayera. Cada prueba fue una ejecución aparte.

Lo que escribimos	Qué pasó
P03;T2;4;320.5;3.6	Se aceptó y aparece en todos los reportes
P01;T2;3;500;4.6	Se aceptó. Ese día P01 pasó de 365 a 865 litros y ganó la bonificación
P03;T2	Sale formato_invalido porque no tiene 5 datos

Lo que escribimos	Qué pasó
P99;T2;4;300;3.6	Se rechaza por productor_desconocido y aparece en R1 (quedan 13 rechazadas)
Solo Enter	No se agrega ninguna entrega
Comprobante de P10	Sale todo en cero
Comprobante de un código que no existe	Dice que el productor no existe y el programa termina normal

```

Ingrese una entrega adicional
(productor;tanque;dia;litros;grasa)
o Enter para omitir: P03;T2
Entrada no registrada: {:error, :formato_invalido}

Reporte 1. Entregas rechazadas

%{productor: "P99", tanque: "T1", dia: 2, litros: 200, grasa: 3.5} -> productor_desconocido
%{productor: "P03", tanque: "T2", dia: 3, litros: 150, grasa: 3.2} -> productor_desconocido
%{productor: "P03", tanque: "T9", dia: 1, litros: 180, grasa: 3.4} -> tanque_desconocido
%{productor: "P06", tanque: "T5", dia: 4, litros: 210, grasa: 3.3} -> tanque_desconocido
%{productor: "P08", tanque: "T1", dia: 7, litros: 210, grasa: 3.3} -> dia_invalido
%{productor: "P10", tanque: "T2", dia: 0, litros: 190, grasa: 3.6} -> dia_invalido
%{productor: "P09", tanque: "T3", dia: "3", litros: 150, grasa: 3.0} -> dia_invalido
%{productor: "P10", tanque: "T1", dia: 3, litros: 950, grasa: 3.5} -> litros_fuera_de_rango
%{productor: "P05", tanque: "T4", dia: 5, litros: 0, grasa: 3.2} -> litros_fuera_de_rango
%{productor: "P06", tanque: "T3", dia: 1, litros: 200, grasa: 16.2} -> porcentaje_invalido
%{productor: "P02", tanque: "T1", dia: 4, litros: 180, grasa: -1.0} -> porcentaje_invalido
%{productor: "P99", tanque: "T9", dia: 9, litros: 900, grasa: 20} -> productor_desconocido

Rechazos por motivo:
productor_desconocido: 3
tanque_desconocido: 2
dia_invalido: 3
litros_fuera_de_rango: 2
porcentaje_invalido: 2

Reporte 2. Ocupación de tanques (de mayor a menor)

```

Captura 7. Al escribir P03;T2 el programa muestra {:error, :formato_invalido} y sigue con los reportes. R1 queda igual, con 12 rechazadas, porque esa línea nunca se convirtió en una entrega.

```
Ingrese el código del productor para el comprobante: P10
```

```
Comprobante de pago
```

```
Productor: Omar Cortés (P10)
```

```
No tuvo entregas válidas en la semana.
```

```
Litros entregados: 0.0 L
```

```
Total entregas: $0
```

```
Total bonificaciones: $0
```

```
Descuento por transporte: $0
```

```
Neto a pagar: $0
```

Captura 8. Omar Cortés (P10) sí existe, pero sus dos entregas fueron rechazadas (una es del día 0 y la otra tiene 950 litros). Por eso su comprobante sale en cero.

Parte D

Bitácora de uso de IA

Integrantes	Herramienta	Errores/Observaciones	Soluciones
Luisa Hernández	Claude	Cómo empezar el proyecto el GitHub para VSCode, ya que siempre habíamos usado IntelliJ	Seguimos los pasos para crear el repositorio y conectarlo a GitHub Desktop.
Isabella Hincapié	Claude	Que nos explicara la liquidación (<code>group_by</code> , <code>Map.get</code> , funciones anónimas)	Con eso reescribimos la liquidación con funciones más cortas y nombres más claros.
Camila González	ChatGPT	Simplificar el código con lo visto en la guía de colecciones	Cambiamos funciones que no vimos en clase (<code>Map.new</code> , <code>map_size</code> , <code>Map.fetch</code>) por otras de la guía.
Camila González	Claude	Encontrar errores al hacer merge, fue un sancocho, profe :(	Básicamente nos tocó hacer push force para volver a versiones anteriores.

Caso de error

Cuando juntamos los archivos de las tres, el programa no corría. Encontramos estos problemas:

1. En `Reportes.exs` escribimos `1..Parametros.dias()`, pero `Parametros.dias()` ya es `1..6`, entonces quedaba `1..(1..6)` y daba error. Lo cambiamos por `Parametros.dias()`.

2. Cambiamos los nombres de las funciones en `Reportes.exs` y `Vista.exs`, pero `main.exs` seguía llamando los nombres viejos (por ejemplo `Reportes.r1`). Tuvimos que actualizar `main.exs`.
3. La validación buscaba los productores en una lista, pero el comprobante los buscaba en un mapa con `Map.get`. Lo cambiamos a `Enum.find` sobre la lista.
4. `Util2.ex` tenía funciones recursivas (ingresar con `:entero`, `:real`, `:boolean` y las de colecciones), y el enunciado no permite recursividad. No las usábamos, pero las quitamos del archivo para no tener problemas.

También tuvimos un error propio: al hacer `litros_por_dia` usamos `Enum.group_by` y después un `Enum.reduce` con `e.litros`. No funcionaba porque al recorrer un mapa cada elemento es una tupla `{dia, lista}` y no tiene `.litros`. Además el `reduce` devolvía un solo número y no un total por día. Lo arreglamos recorriendo cada día y sumando su lista.

Reflexión

Durante el proceso del pensamiento computacional nos encontramos varios baches a la hora de cada una entender los requerimientos de proyecto, una vez estando en la misma página buscamos una manera de conectar directamente el repositorio con lo que cada una íbamos trabajando y luego poder hacer merge, así que utilizamos la herramienta de GitHub Desktop para apoyarnos en ese sentido, aunque tuvimos ciertos inconvenientes ya en el momento de cada una codificar y que al unirlos ambos commits se repelen entre si podemos solucionarlo con tiempo, incluido el que mencionamos en la bitácora.

Lo que respecta al proyecto en si, nos esforzamos en aplicar los conocimientos adquiridos a través de las clases del semestre, sin embargo con la ayuda de la IA pudimos afianzar ciertos conceptos o formas más prácticas de resolver cada uno de los requerimientos del parcial, también entre las tres pudimos apoyarnos en las fortalezas de la otra y lograr el mejor resultado posible.